In [147…
```
# import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

In [4]:
```
# Excel file path
excel_file = 'customer_churn_data_raw.xlsx'

# Create SQLite database connection
conn = sqlite3.connect('customer_churn.db')

# Read all sheet names from Excel file
excel_data = pd.ExcelFile(excel_file)

# Loop through each sheet and store as separate table
for sheet in excel_data.sheet_names:
    df = pd.read_excel(excel_file, sheet_name=sheet) # Read sheet into dataframe
    # Write dataframe to SQLite table
    df.to_sql(
        name=sheet,          # Table name = Sheet name
        con=conn,
        if_exists='replace', # Replace table if already exists
        index=False
    )

# Close connection
conn.close()

print("All sheets successfully converted into SQLite tables.")
```

```
All sheets successfully converted into SQLite tables.
```

In [5]:
```
# Connect to SQLite database
conn = sqlite3.connect('customer_churn.db')

# sql query to Get all table names
sql_query = """
        SELECT name
        FROM sqlite_master
        WHERE type='table';
"""

# read sql query in pandas
tables = pd.read_sql(sql_query, conn)

# create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn) # Read table into datafra
    globals()[f"df_{table_name}"] = df                    # Create dynamic datafram
    print(f"Created dataframe: df_{table_name}")
```

```
# Close connection
conn.close()
```

```
Created dataframe: df_db_customer
Created dataframe: df_db_subscription
Created dataframe: df_db_support
```

In [6]:
```
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")
    # Get column information
    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print("Columns:")
    print(columns['name'].tolist())

# Close connection
conn.close()
```

```
Table Name: db_customer
Columns:
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name: db_subscription
Columns:
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan
_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charge
s', 'cltv', 'churn_score']

Table Name: db_support
Columns:
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

In [7]:
```
df_db_customer.tail()
```

Out[7]:

	customerid	name	country	state	gender	dob	interests	pincode
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

In [8]:
```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   customerid  21 non-null     str
 1   name        21 non-null     str
 2   country     18 non-null     str
 3   state       21 non-null     str
 4   gender      21 non-null     str
 5   dob         21 non-null     str
 6   interests   4 non-null      str
 7   pincode     0 non-null      object
dtypes: object(1), str(7)
memory usage: 2.5+ KB
```

In [9]:
```
df_db_customer.rename(columns = {'name' : 'customer_name'}, inplace= True)
df_db_customer
```

Out[9]:

	customerid	customer_name	country	state	gender	dob	interests	pincod
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	Non
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	Non
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	Non
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	Non
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	Non
5	0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00	NaN	Non
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00	NaN	Non
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00	NaN	Non
8	0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00	NaN	Non
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00	NaN	Non
10	0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00	NaN	Non
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00	NaN	Non
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00	NaN	Non
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00	NaN	Non

	customerid	customer_name	country	state	gender	dob	interests	pincod
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00	NaN	Non
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00	job	Non
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	Non
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	Non
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	Non
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	Non
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	Non

In [10]:
```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   customerid     21 non-null     str
 1   customer_name  21 non-null     str
 2   country        18 non-null     str
 3   state          21 non-null     str
 4   gender         21 non-null     str
 5   dob            21 non-null     str
 6   interests      4 non-null      str
 7   pincode        0 non-null      object
dtypes: object(1), str(7)
memory usage: 2.5+ KB
```

In [11]:
```
df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

In [12]:
```
df_db_customer['gender'] = df_db_customer['gender'].replace({'Men' : 'Male ', 'Woma
```

In [13]:
```
df_db_customer['gender'].unique()
```

Out[13]:
```
<ArrowStringArray>
['Male', 'Female', 'Women', 'Male ']
Length: 4, dtype: str
```

In [14]:
```
df_db_customer[df_db_customer['country'].isna()]
```

Out[14]:

	customerid	customer_name	country	state	gender	dob	interests	pincode
5	0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10	NaN	None
8	0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07	NaN	None
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01	NaN	None

In [15]:
```
state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('state'

df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['state'
```

In [16]:
```
df_db_customer[df_db_customer['country'].isna()]
```

Out[16]:

	customerid	customer_name	country	state	gender	dob	interests	pincode

In [17]:
```
df_db_customer[['country' , 'state']]
```

Out[17]:

	country	state
0	India	Maharashtra
1	India	Karnataka
2	India	Delhi
3	India	Nagaland
4	India	Delhi
5	India	Delhi
6	India	Meghalaya
7	India	Rajasthan
8	Nepal	Kathmandu
9	Nepal	Kathmandu
10	India	Maharashtra
11	India	Karnataka
12	India	Telangana
13	India	Meghalaya
14	India	Uttar Pradesh
15	India	Delhi
16	India	Meghalaya
17	India	Rajasthan
18	India	Telangana
19	India	Uttar Pradesh
20	India	Maharashtra

In [18]:
```
df_db_subscription.head()
```

Out[18]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	M
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	A
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	M

In [19]:
```
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   customerid              21 non-null     str
 1   subscription_start_date 21 non-null     str
 2   subscription_type       21 non-null     str
 3   renewal_date            21 non-null     str
 4   plan_type               21 non-null     str
 5   contract_type           21 non-null     str
 6   cancellation_date       6 non-null      str
 7   cancellation_reason     6 non-null      str
 8   monthly_charges         21 non-null     float64
 9   cltv                    21 non-null     int64
 10  churn_score             21 non-null     int64
dtypes: float64(1), int64(2), str(8)
memory usage: 3.1 KB
```

In [20]:
```
date_col = ['subscription_start_date',  'renewal_date', 'cancellation_date']

df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     str
 1   subscription_start_date  21 non-null     datetime64[us]
 2   subscription_type        21 non-null     str
 3   renewal_date             21 non-null     datetime64[us]
 4   plan_type                21 non-null     str
 5   contract_type            21 non-null     str
 6   cancellation_date        6 non-null      datetime64[us]
 7   cancellation_reason      6 non-null      str
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 2.7 KB
```

In [21]:
```
df_db_support.head()
```

Out[21]:

	customerid	complaint_date	escalations	csat_score	col_1	comment
0	0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
1	0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
2	0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	NaN
3	0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
4	0013-SMEOE	2024-11-01 00:00:00	N	30	None	NaN

In [22]:
```
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      str
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
 4   col_1           0 non-null      object
 5   comment         4 non-null      str
dtypes: int64(1), object(1), str(4)
memory usage: 895.0+ bytes
```

In [23]:
```
df_db_support.drop(columns=['col_1', 'comment'], inplace=True)
```

In [24]:
```
df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])

df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      datetime64[us]
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 519.0 bytes
```

In [25]:
```
df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_date']
```

In [26]:
```
df_db_subscription.head()
```

Out[26]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	M
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	A
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	M

In [27]:
```
df = (df_db_subscription
    .merge(df_db_customer, on = 'customerid', how= 'left')
    .merge(df_db_support, on = 'customerid', how= 'left') )
```

In [28]:
```
df.shape
```

Out[28]: (23, 22)

In [29]:
```
df_db_subscription['customerid'].nunique()
```

Out[29]: 21

In [30]:
```
df_db_customer['customerid'].nunique()
```

Out[30]: 21

In [31]:
```
df_db_support['customerid'].size
```

Out[31]: 9

In [32]:
```
df_db_support
```

Out[32]:

	customerid	complaint_date	escalations	csat_score
0	0003-MKNFE	2024-08-28	N	60
1	0003-MKNFE	2024-08-28	Y	10
2	0013-EXCHZ	2024-01-20	Y	20
3	0013-MHZWF	2025-03-18	N	90
4	0013-SMEOE	2024-11-01	N	30
5	0017-IUDMW	2024-04-10	Y	25
6	0019-EFAEP	2024-09-27	Y	30
7	0022-TCJCI	2024-09-13	Y	10
8	0022-TCJCI	2024-09-14	N	90

In [33]:
```
df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customerid'
```

In [34]:
```
df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('custom
```

In [35]:
```
df_db_support['customerid'].size
```

Out[35]: 7

In [36]:
```
df = (df_db_subscription
    .merge(df_db_customer, on = 'customerid', how= 'left')
    .merge(df_db_support, on = 'customerid', how= 'left') )
```

In [37]:
```
df.shape
```

Out[37]: (21, 23)

In [38]:
```
df.to_csv('exported_churn_data.csv', index=False)
```

In [39]:
```
df.columns
```

Out[39]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'interests', 'pincode', 'complaint_date', 'escalations', 'csat_score',
       'complaint_count'],
      dtype='str')
```

In [40]:
```
churn_rate = df['churn_flag'].mean()*100
print("Churn Rate = ", round(churn_rate,2), "%")
```

```
Churn Rate =  28.57 %
```

In [41]:
```
retention_rate = 100 - churn_rate
print("Retention Rate = ", round(retention_rate,2), "%")
```

```
Retention Rate =  71.43 %
```

In [42]:
```
churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).res
print(churn_by_plan)
```

```
  plan_type  churn_rate_pct
0     Basic           60.00
1   Premium           14.29
2  Standard           22.22
```

In [43]:
```
churn_by_state = (df.groupby('state')['churn_flag'].mean().mul(100).round(2).reset_
print(churn_by_state)
```

```
           state  churn_rate_pct
0          Delhi           25.00
1      Karnataka          100.00
2      Kathmandu            0.00
3    Maharashtra            0.00
4      Meghalaya           66.67
5       Nagaland            0.00
6      Rajasthan            0.00
7      Telangana           50.00
8  Uttar Pradesh            0.00
```

In [44]:
```
churn_by_subscription_type = (df.groupby('subscription_type')['churn_flag'].mean().
print(churn_by_subscription_type)
```

```
  subscription_type  churn_rate_pct
0           Organic            0.00
1              Paid           16.67
2          Refferal           83.33
```

In [45]:
```
# ARPU
arpu = df['monthly_charges'].mean()
print('ARPU = ', round(arpu,2))
```

```
ARPU =  18.85
```

In [46]:
```
df.head(2)
```

Out[46]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A

2 rows × 23 columns

In [47]:
```
df['tenure_days'] = np.where(
        df['cancellation_date'].notna(),
    (df['cancellation_date'] - df['subscription_start_date']).dt.days,

    (today - df['subscription_start_date']).dt.days
)
```

```
avg_tenure = df['tenure_days'].mean()
print("Avg Tenure (Days) = ", round(avg_tenure),0)
```

```
Avg Tenure (Days) =  1553 0
```

In [48]:
```
revenue_at_risk = df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
print("Revenue at Risk (Rs 'K') =", revenue_at_risk)
```

```
Revenue at Risk (Rs 'K') = 73.94
```

In [49]:
```
escalation_rate = (df['escalations']=='Y').mean()*100
print("Escalation Rate =", round(escalation_rate, 2), "%")
```

```
Escalation Rate = 19.05 %
```

In [50]:
```
df['complaint_count'].sum() / df['customerid'].nunique()
print("Avg Complaints per User = ", round(avg_complaints, 2))
```

```
Avg Complaints per User =  0.43
```

In [51]:
```
df['escalations'] = np.where(df['escalations'] == 'Y', 1, 0)
corr_df = df[['escalations', 'churn_flag']].dropna()


correlation = corr_df['escalations'].corr(df['churn_flag'])
print("Correlation between escalations vs churn is ", round(correlation,2))
```

```
Correlation between escalations vs churn is  0.77
```

In [52]:
```
conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >= 50) & (df['churn_score'] < 70),
    (df['churn_score'] >= 70)
]

choices = ['low', 'med', 'high' ]

df['churn_risk'] = np.select(conditions, choices, default='unknown')
```

In [53]:
```
df[['churn_risk', 'churn_score']].tail()
```

Out[53]:

	churn_risk	churn_score
16	med	62
17	low	27
18	high	99
19	low	7
20	low	47

In [67]:
```
df.head()
```

Out[67]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	M
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	A
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	M

5 rows × 25 columns

In [72]:
```
df_visuals = df.copy()
```

In [81]:
```
df_visuals.columns
```

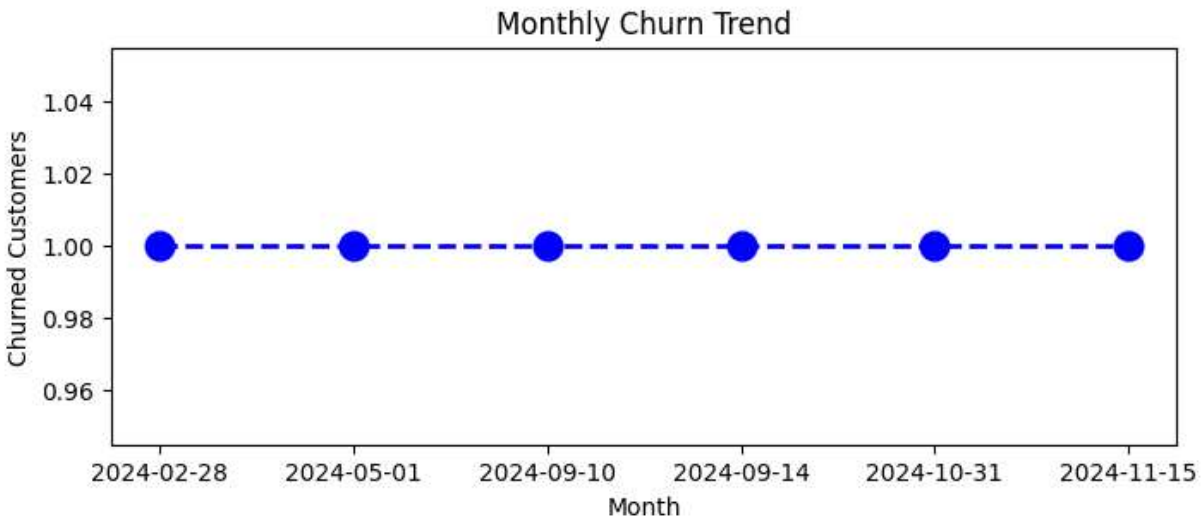
Out[81]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'interests', 'pincode', 'complaint_date', 'escalations', 'csat_score',
       'complaint_count', 'tenure_days', 'churn_risk'],
      dtype='str')
```

In [97]:
```
df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')

churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_date').

plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values,   color='blue', marker=

plt.title('Monthly Churn Trend')
plt.ylabel('Churned Customers')
plt.xlabel('Month')
plt.show()
```

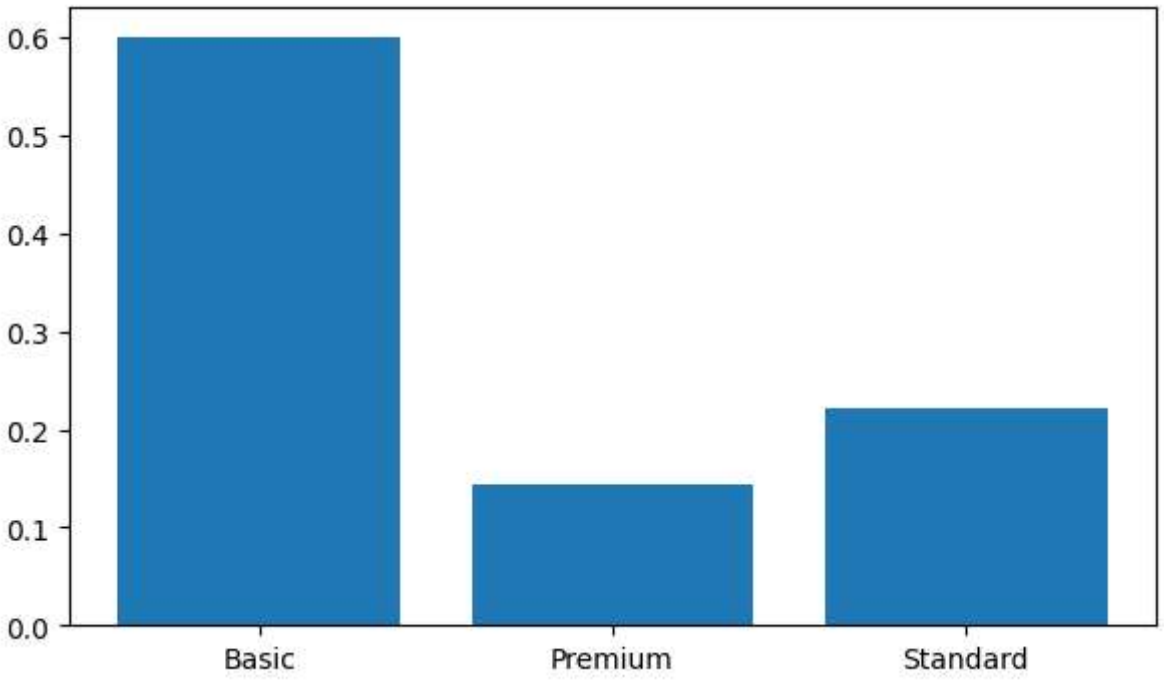


In [101…
```
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()



plt.figure(figsize=(7, 4))

plt.bar(churn_plan.index, churn_plan.values)
plt.show()
```

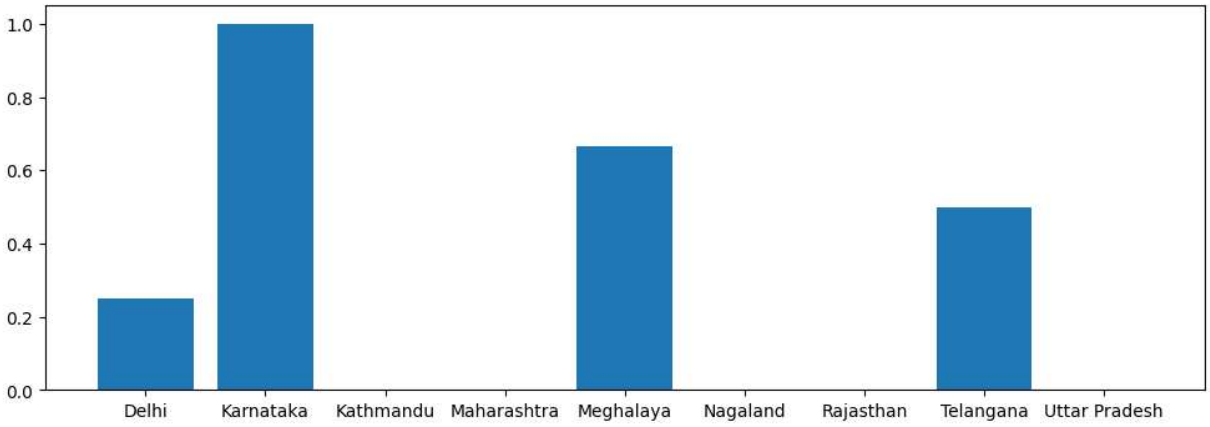


In [103…
```
churn_plan = df_visual.groupby('state')['churn_flag'].mean()



plt.figure(figsize=(12, 4))

plt.bar(churn_plan.index, churn_plan.values)
plt.show()
```



In [104…
```
#df_encoded =
df[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escal
```

Out[104…

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	Standard	Annual	12	0	low	0
1	Premium	Annual	91	1	high	0
2	Basic	Monthly	34	0	low	0
3	Premium	Annual	8	0	low	0
4	Standard	Monthly	88	1	high	0

In [112…
```
df_encoded.head()
```

Out[112…

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	2	0	12	0	1	0
1	1	0	91	1	0	0
2	0	1	34	0	1	0
3	1	0	8	0	1	0
4	2	1	88	1	0	0

In [110…
```
import warnings
warnings.filterwarnings("ignore")

# encoding based on numbes

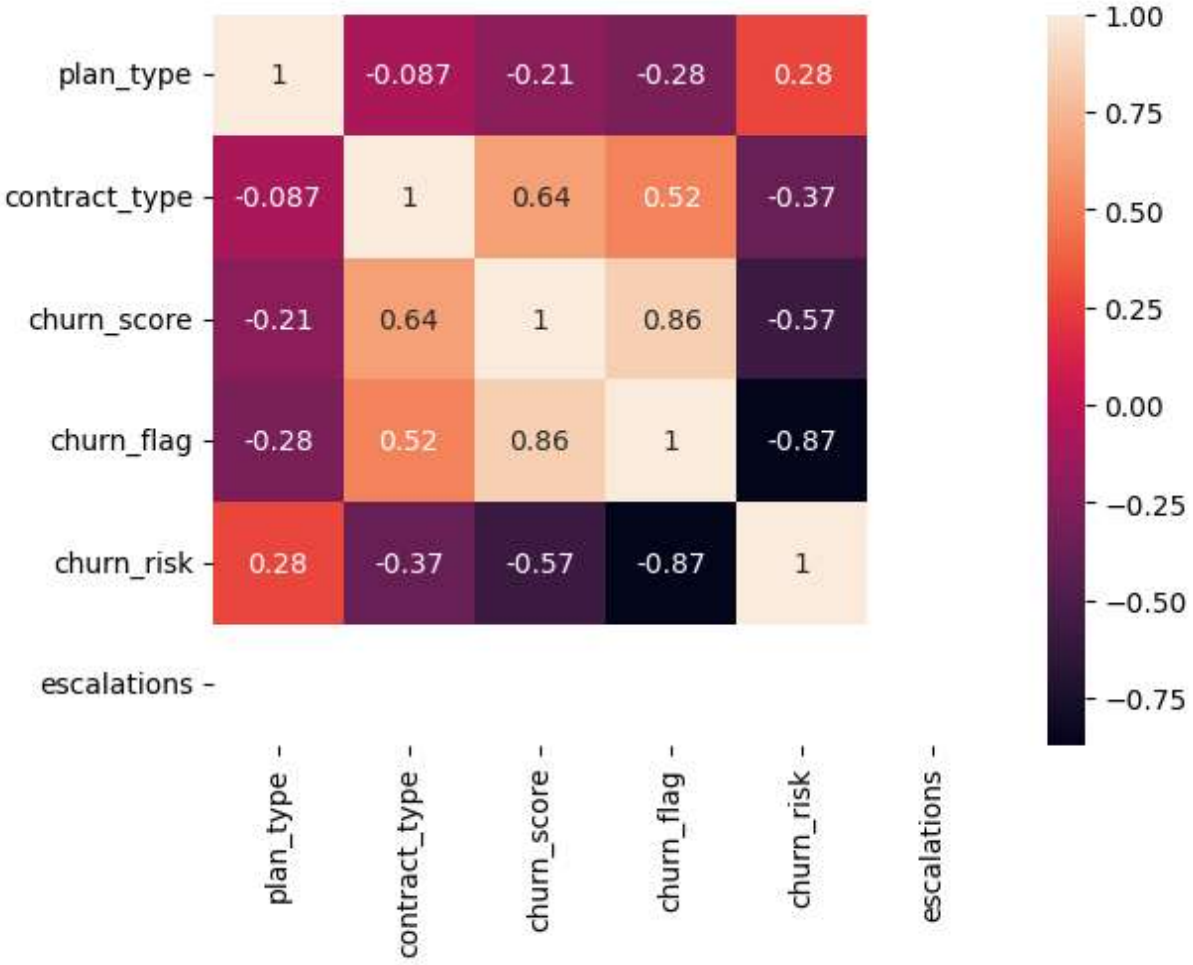
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag',

categorial_cols = ['plan_type', 'contract_type', 'churn_risk']

for col in categorial_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

In [111…
```
sns.heatmap(df_encoded.corr(), annot=True)
```

Out[111… <Axes: >



In [115…
```
# encoding based on priority

df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag',

order_mappings = {
    'plan_type' : ['Basic', 'Standard', 'Premium'],
    'contract_type' : ['Monthly', 'Annual'],
    'churn_risk' : ['low', 'med', 'high']
    }

for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categories
```

In [117…
```
df_encoded.head()
```

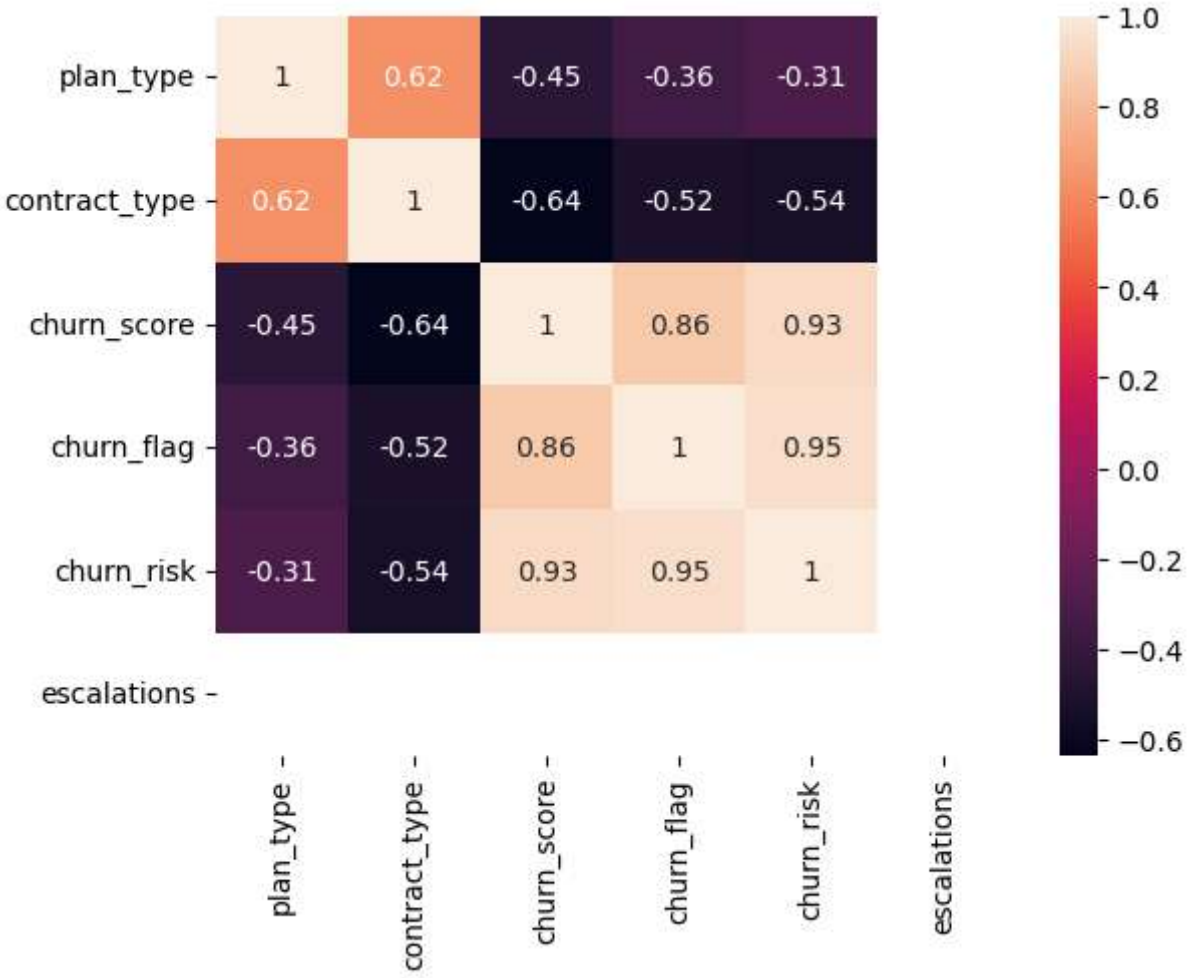
Out[117…

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	1	1	12	0	0	0
1	2	1	91	1	2	0
2	0	0	34	0	0	0
3	2	1	8	0	0	0
4	1	0	88	1	2	0

In [118…

```
sns.heatmap(df_encoded.corr(), annot=True)
```

Out[118… <Axes: >



In [119…

```
# Heatmap Using Matplotlib - difficult to plot it

corr_matrix = df_encoded.corr() # Correlation matrix

fig, ax = plt.subplots(figsize=(10, 6)) # Create figure

cax = ax.imshow(corr_matrix, cmap='coolwarm') # Heatmap

fig.colorbar(cax) # Add colorbar
```

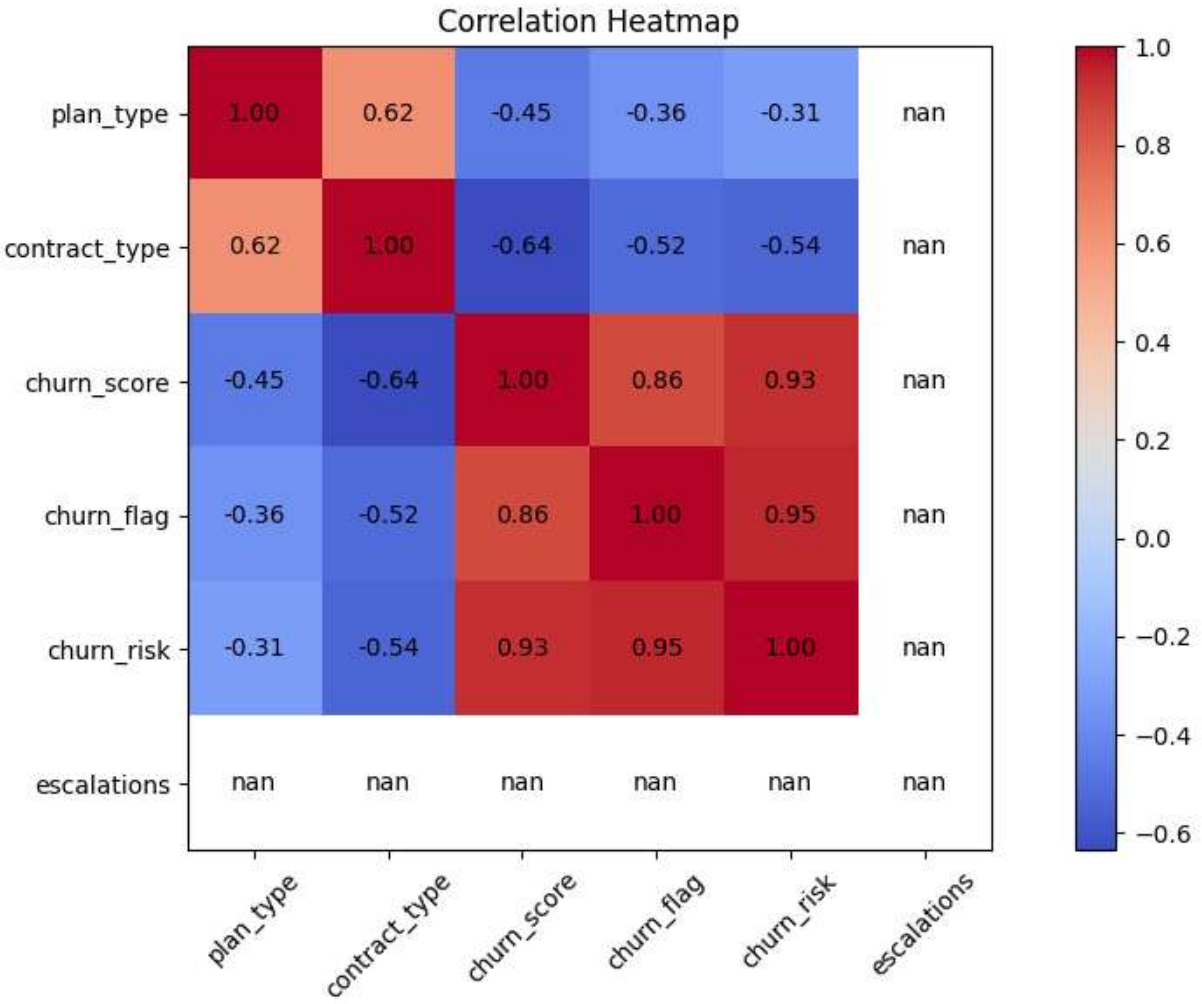
```
# Axis labels
ax.set_xticks(np.arange(len(corr_matrix.columns)))
ax.set_yticks(np.arange(len(corr_matrix.columns)))

ax.set_xticklabels(corr_matrix.columns, rotation=45)
ax.set_yticklabels(corr_matrix.columns)

# Annotate values inside cells
for i in range(len(corr_matrix.columns)):
    for j in range(len(corr_matrix.columns)):
        ax.text(
            j,
            i,
            f"{corr_matrix.iloc[i, j]:.2f}",
            ha='center',
            va='center'
        )

plt.title('Correlation Heatmap') # Title

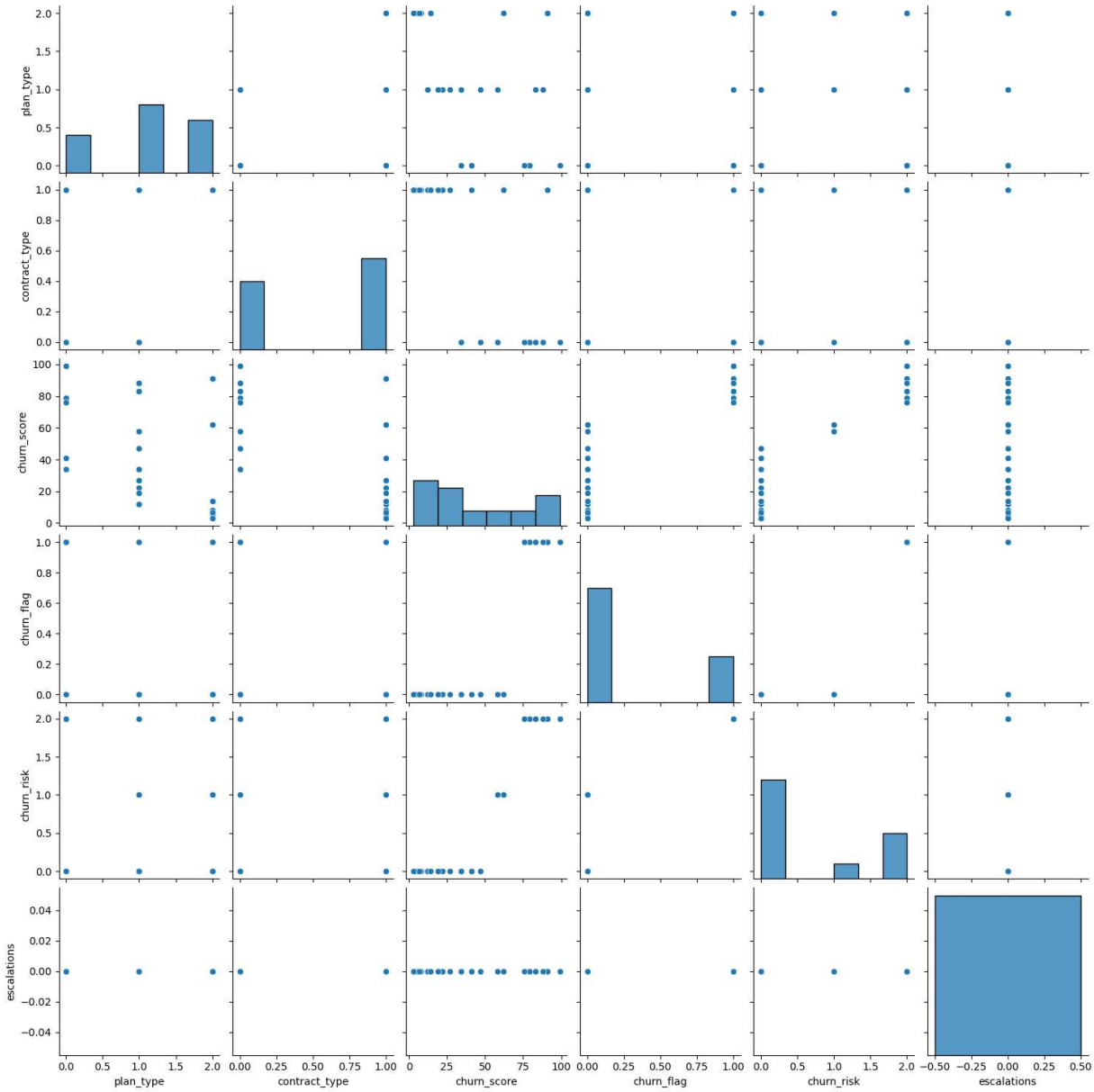
plt.tight_layout()
plt.show()
```



In [120…

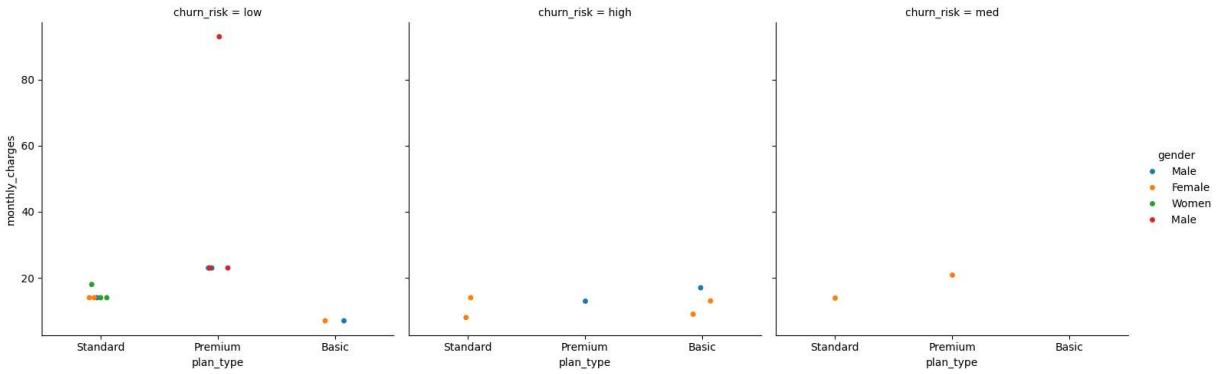
```
# pairplot
sns.pairplot(df_encoded)
```

Out[120… <seaborn.axisgrid.PairGrid at 0x1c71ca8bcb0>



In [122…
```
# categorical
sns.catplot(data=df_visual,
    x='plan_type',
    y='monthly_charges',
    hue='gender',
    col='churn_risk')
```

Out[122… <seaborn.axisgrid.FacetGrid at 0x1c777a5cc20>



In [172…
```
import pandas as pd
import sqlite3 as sq

# create db in sql
conn = sqlite3.connect('my_database.sqlite')

#table details
conn.execute("CREATE TABLE users (first_name TEXT, country TEXT, budget INTEGER)")

# commit and save
conn.commit()

print("Table Created Successfully!")
```

```
Table Created Successfully!
```

In [173…
```
# insert data

cursor = conn.cursor()

# write sql query to insert records in sql table
cursor.execute(
    """
        INSERT INTO users VALUES
            ('Madhav', 'India', 5000),
            ('Rishabh', 'Germany', 2500),
            ('Vishakha', 'India', 3500)
    """
)

# commit and save
conn.commit()


print("Data Inserted successfully!")
```

```
Data Inserted successfully!
```

In [174…
```
# check inserted data in table
conn = sqlite3.connect('my_database.sqlite')
query = """SELECT * FROM users"""

df_results = pd.read_sql(query, conn)
```

```
df_results.head()
```

Out[174…

	first_name	country	budget
0	Madhav	India	5000
1	Rishabh	Germany	2500
2	Vishakha	India	3500

In [177…

```
# aggregation
query ="""
        SELECT country, SUM(budget) as total_budget
        FROM users
        GROUP BY country
"""

df_agg = pd.read_sql(query, conn)
df_agg
```

Out[177…

	country	total_budget
0	Germany	2500
1	India	8500

In [178…

```
conn.close()
```

In [ ]: